DRAFT — HelixConstitution §11.4.107 Anti-Forgetting Enforcement Anchor

Status: DRAFT — NOT YET LANDED IN constitution/Constitution.md
Prepared by: Stream B subagent, 2026-06-02
Classification: universal (§11.4.17)
CONST-049 pipeline: see Part B below
BLOCKED-UNTIL: see Part C below

Part A — Proposed §11.4.107 Clause Text

---

§11.4.107 — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)

Short tag: `anti-forgetting-enforcement`.

Forensic anchor — operator mandate:

> UNCONFIRMED: pending operator's verbatim anti-forgetting mandate quote (tracked: PENDING item 3 in the current session's CONTINUATION.md handoff).

Background context (UNCONFIRMED quote omitted; factual description only): the motivating incident is documented in `docs/AGENT_GUARDRAILS.md` and in `scripts/hooks/guard-forbidden-commands.sh`'s in-source header. During an on-device-API build, emulator subagents ran raw host-direct `emulator`/`adb` commands instead of routing through the Containers submodule as required by the governing clauses (§6.X / §6.V / §6.AG in the consuming-project layer). The root cause was that the orchestrator **forgot to inject the rule into subagent prompts**. A rule the orchestrator forgets to paste is not enforcement — it is a social contract with the agent's memory, and that contract is broken by every cold-session start, context-window exhaustion, and sub-agent dispatch. The mechanical fix is twofold:

(1) a **PreToolUse guard hook** that blocks the forbidden command classes at the tool-call boundary unconditionally, and (2) a **canonical subagent preamble document** that the orchestrator pastes verbatim into every dispatch, plus a **pre-action checklist** for the orchestrator's own actions. Together these are the "anti-forgetting" enforcement layer: the hook is the floor (commands cannot be executed even if the agent forgot every rule), the preamble is the ceiling (the full live ruleset, not just what the hook can pattern-match). This two-layer model was introduced in the Lava project (commit family on `master`, 2026-06-01/02) and is explicitly designed to be universal — `scripts/hooks/guard-forbidden-commands.sh` carries `Classification: universal` in its header and is portable by design (no Lava-specific paths, no `jq` hard dependency).

Rule. Every HelixConstitution-consuming project MUST deploy the §11.4.107 anti-forgetting enforcement layer consisting of three components, all maintained together:

(A) PreToolUse guard hook — the mechanical floor.

A shell script MUST be wired as a `PreToolUse` hook in the AI agent runtime's project-scoped settings file (e.g. `.claude/settings.json` for Claude Code) to intercept every `Bash` tool call before execution and BLOCK the forbidden command classes at the tool-call boundary. The hook:

1. Reads the tool invocation as JSON from stdin (`.tool_name` + `.tool_input.command`).
2. Exits 0 for non-Bash tools (pass-through) and empty/missing commands.
3. For Bash tool calls, pattern-matches against the following mandatory blocked classes, each mapped to its governing clause:

Blocked class	Governing clause(s)	Examples
Raw host-direct Android emulator launch / APK install / instrumentation run	§11.4.76 (Containers mandate) + any project-layer emulator clause (e.g. §6.X / §6.V / §6.AG in Lava)	`emulator -avd …`, `$ANDROID_HOME/emulator/emulator …`, `adb install …`, `adb -s … install …`, `am instrument …`
Force-push / hook bypass / signature bypass	§11.4.41 (pre-force-push merge-first) + §11.4.71 (pre-push fetch) +	`git push --force`, `git push -f`, `git push --force-with-lease`, `--no-verify`, `--no-gpg-sign`

Blocked class	Governing clause(s)	Examples
	project-layer §6.T.3 equivalent	
Privilege escalation	§11.4 no-sudo (project-layer §6.U equivalent)	`sudo …`, `su`, `su -`, `su -l …`
Host power management	§12 host-session-safety (all consuming projects)	`systemctl suspend/hibernate/poweroff/reboot/halt/kill-user/kill-session`, `loginctl …`, `pm-suspend/hibernate`, `shutdown …`

4. Exits 2 (block) when a match is found; the stderr text printed is fed back to the agent as the refusal reason, citing the governing clause.

5. Supports a documented-exception escape hatch: a command containing the literal marker `# guardrails:allow <reason>` is WARNED (stderr) but NOT blocked — EXCEPT for host-power commands, which are categorically non-overridable.

The **canonical implementation** of this hook lives at `constitution/scripts/hooks/guard-forbidden-commands.sh` in this constitution submodule. Consuming projects MUST reference it at that path (inherited by reference per §11.4.80) — NEVER copy it locally (a copy diverges silently). The script is portable: no `jq` hard dependency (built-in awk fallback), no consumer-project-specific paths, no hardcoded credentials.

Anti-bluff requirement: a hermetic test suite (at minimum 20 cases) MUST verify every blocked class exits 2, every allowed command exits 0, the escape hatch fires for non-power classes, and the host-power class rejects even with the escape marker. Paired §1.1 mutation: remove the `emulator -avd` pattern from the hook → the emulator-gate case exits 0 → test FAILs → restore → test PASSes.

(B) Subagent Constitutional Preamble — the semantic ceiling.

A canonical document MUST exist at `docs/AGENT_GUARDRAILS.md` (or the project's equivalent governance preamble path, recorded in the project's `CLAUDE.md`) containing:

1. **The SUBAGENT CONSTITUTIONAL PREAMBLE block** — a verbatim, self-contained text block covering the top-10 mandatory constraints that cannot be pattern-matched by the hook alone (anti-bluff intent, resource caps, evidence honesty, continuation maintenance, hardcoding prohibition, distribute gate, remote policy, no-guessing vocabulary, etc.)

plus the constraint classes that the hook already enforces (emulators, force-push, sudo, host-power) for completeness.
2. **Clear instruction that the orchestrator MUST paste this block verbatim into every subagent dispatch.**

The preamble is the semantic complement to the hook: the hook catches command-class violations; the preamble pre-informs the agent of all rules it must follow before it even issues a command. Together they eliminate the "agent forgot because the orchestrator didn't paste it" failure class.

Anti-bluff requirement: the preamble MUST be structured so it can be mechanically verified to be present in `docs/AGENT_GUARDRAILS.md` by `check-constitution.sh` (or the project-equivalent sweep); the document MUST contain the anchor literal `11.4.107` once this clause lands. Paired §1.1 mutation: remove the preamble's hook-reference sentence → constitution checker detects absence → FAIL.

(C) Orchestrator Pre-Action Checklist — guard against self-forgetting.

The same `docs/AGENT_GUARDRAILS.md` document MUST contain an **ORCHESTRATOR PRE-ACTION CHECKLIST** covering at minimum:

- Before any subagent dispatch: confirm the preamble (sub-rule B) is pasted verbatim.
- Before any emulator/device action: confirm the run routes through the Containers submodule CLI, not host-direct; confirm no live device is targeted; confirm the gate host is eligible (otherwise honestly BLOCKED).
- Before any distribute action: confirm Challenge Tests EXECUTED (not compiled) against the exact artifact; version code bumped; CHANGELOG entry present; debug-stage evidence present if two-stage distribute applies.
- Before any push / destructive git action: confirm no force flags without per-operation operator approval; remote is the approved set only; for history rewrite, hardlinked `.git` backup made first.
- Before any host-affecting command: confirm the command is not in the host-power blocked class.

Anti-bluff requirement: the checklist MUST be present in the preamble document and verifiable by `check-constitution.sh`. Each checklist item is traceable to a governing clause cited inline.

Mechanical enforcement.

Consuming-project `check-constitution.sh` (or `verify-all-constitution-rules.sh`) MUST verify ALL of:

1. `constitution/scripts/hooks/guard-forbidden-commands.sh` exists at the canonical path in the constitution submodule.

2. The consumer's AI-agent settings file (`.claude/settings.json` or equivalent) contains a `PreToolUse` hook entry referencing the canonical script path.
3. `docs/AGENT_GUARDRAILS.md` (or the equivalent preamble path) exists, contains the `SUBAGENT CONSTITUTIONAL PREAMBLE` heading, and contains the `ORCHESTRATOR PRE-ACTION CHECKLIST` heading.
4. A hermetic test for the hook exists (at minimum, a file under `tests/hooks/` or equivalent) and passes.
5. The anchor literal `11.4.107` is present in every per-scope governance file (`CLAUDE.md`, `AGENTS.md`, `QWEN.md`) of the consuming project and its owned submodules per §11.4.35 inheritance.

Pre-build gate `CM-ANTI-FORGETTING-ENFORCEMENT` enforces (1)–(4). Propagation gate `CM-COVENANT-114-107-PROPAGATION` enforces (5). Paired §1.1 meta-test mutations: (a) remove the `PreToolUse` hook entry from the agent settings file → gate (2) FAILs; (b) delete `docs/AGENT_GUARDRAILS.md` → gate (3) FAILs; (c) remove the canonical hook script from the constitution submodule → gate (1) FAILs; (d) strip the `11.4.107` literal from a consumer `CLAUDE.md` → propagation gate FAILs.

Inheritance.

Applies recursively to every submodule, every feature, every new artifact, every project consuming this constitution. Submodule constitutions MAY add stricter rules (additional blocked command classes, additional checklist items, stricter test counts) but MUST NOT relax any clause. Per §11.4.35: this universal clause lives in the constitution submodule; consuming projects' `CLAUDE.md` / `AGENTS.md` / `QWEN.md` carry the inheritance pointer-block per §11.4.35 + the §11.4.107 literal per the propagation gate.

The principle this clause enshrines (in the fewest possible words): A constraint that depends on an agent remembering it is not a constraint — it is a hope. The PreToolUse hook converts the most dangerous command classes into hard-blocked failures regardless of agent memory. The preamble document converts the full ruleset into something the orchestrator can inject rather than recite from training. Together they are the minimum viable "anti-forgetting" enforcement posture for any AI-agent-assisted project under this constitution.

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant — the hook and preamble are the floor and ceiling of the anti-bluff enforcement layer), §11.4.6 (no-guessing — the escape hatch requires a `<reason>` exactly because undocumented exceptions are a guessing posture), §11.4.10 (credentials — the host-power class is categorically non-overridable, analogous to credential leaks), §11.4.75 (mechanical enforcement — §11.4.107 is the AI-agent-side specialisation of §11.4.75's git-hook mechanical enforcement; together they cover the full automated-and-agentic surface), §11.4.76 (Containers mandate — the emulator gate in the

hook enforces §11.4.76 at the tool-call boundary), §11.4.78 / §11.4.79 / §11.4.80 (CodeGraph — the hook passes codegraph MCP calls through untouched; the preamble includes the anti-forgetting constraint about CodeGraph use), §11.4.81 (cross-platform parity — the hook script is portable bash/awk with no OS-specific primitives; per-OS blocked-class equivalents are addable via consuming-project extension), §11.4.84 (working-tree quiescence — the pre-action checklist instructs agents to grep for mutation markers before staging), §11.4.98 (full-automation mandate — the hook's test suite MUST itself be fully self-driving end-to-end), §11.4.102 (systematic-debugging — any hook violation that surfaces should trigger the systematic-debugging arc, not a quick workaround), §12 (host-session-safety — the host-power class is non-overridable; §12 is the constitutional root of that prohibition).

Classification: universal (§11.4.17) — the PreToolUse guard hook + subagent preamble + orchestrator checklist pattern is vendor-neutral (defined against the JSON stdin contract that every Claude Code hook receives; the pattern ports to any agent runtime that exposes a pre-tool-call interception point) and reusable across ANY project that uses AI coding agents. The specific blocked command-class list at the UNIVERSAL layer covers the four classes that are unconditionally forbidden by this constitution (host-power, privilege escalation, force-push/bypass, raw emulator host-direct); consuming projects extend the list with project-specific additions per §11.4.35.

Non-compliance is a release blocker. No escape hatch — no `--skip-pretooluse-hook`, `--no-guardrails-doc`, `--anti-forgetting-optional`, `--single-layer-sufficient` flag exists. A project whose agent can forget a constitutional constraint and execute the forbidden command is constitutionally undefended, irrespective of how well the docs are written.

Canonical authority: this Constitution.md §11.4.107 in the HelixConstitution submodule; reference implementation `constitution/scripts/hooks/guard-forbidden-commands.sh` + reference preamble document `constitution/docs/AGENT_GUARDRAILS.md` (path within the submodule). Consuming projects inherit by reference per §11.4.80.

Part B — CONST-049 Landing Plan (7-Step Pipeline)

The CONST-049 pipeline for a constitution submodule change is:

1. **Fetch + pull --ff-only first (§11.4.26 step 1 / §11.4.37).** Inside the `./constitution/` worktree:

```
git -C constitution fetch --all --prune
git -C constitution pull --ff-only origin main
# Verify clean: git -C constitution status
```

If the pin is behind upstream (check `git -C constitution log HEAD..origin/main --oneline`), integrate before editing. The constitution pin at the parent level MUST be advanced in the same commit that lands the clause (step 7).

2. **§11.4.32 verify-all sweep (pre-edit baseline).**

```
bash scripts/verify-all-constitution-rules.sh
# Must exit 0 (all gates PASS) before any edit to
        constitution/.
```

3. **Add the clause AND lift the guard script.**

   - Append `### §11.4.107 — ...` to `constitution/Constitution.md` immediately after the §11.4.106 block (before `## §12.`).
   - Mirror the summary entry into `constitution/CLAUDE.md`, `constitution/AGENTS.md`, `constitution/QWEN.md` (the house style is: a brief paragraph + the `CM-COVENANT-114-107-PROPAGATION` gate reference + `Canonical authority` line — see §11.4.106 CLAUDE.md mirror for the exact format).
   - Lift `scripts/hooks/guard-forbidden-commands.sh` to `constitution/scripts/hooks/guard-forbidden-commands.sh` (the `constitution/scripts/` directory already exists per §11.4.80 `codegraph_update.sh` / `codegraph_sync.sh` precedent). The file is identical to the consuming-project version — it was written to be portable. Update `.mcp.json` / `.claude/settings.json` hook path references to point at the constitution-relative path for any project that inherits by reference.
   - Lift `docs/AGENT_GUARDRAILS.md` to `constitution/docs/AGENT_GUARDRAILS.md` as the canonical reference document (consuming projects EITHER inherit this file by reference OR maintain their own at `docs/AGENT_GUARDRAILS.md` with an inheritance pointer).
   - Update `constitution/docs/CODEGRAPH.md` if the constitution submodule maintains one, to reflect the new script location.

4. **Regenerate exports (§11.4.65 universal markdown export).** The constitution submodule's `constitution/Constitution.md` and the CLAUDE/AGENTS/QWEN mirrors gain new content; their `.html` + `.pdf` siblings MUST be regenerated:

```
cd constitution
pandoc Constitution.md -o Constitution.html
pandoc Constitution.md -o Constitution.pdf  # or weasyprint
        Constitution.html Constitution.pdf
# Repeat for CLAUDE.md, AGENTS.md, QWEN.md mirrors
```

5. **Commit + push to ALL 4 HelixConstitution upstreams.** HelixConstitution is HelixDevelopment-owned and maintains 4 upstreams (GitHub + GitLab + GitFlic + GitVerse) per the §6.AD.1 carve-out — unlike Lava's §6.W 2-mirror constraint. Stage ONLY the governance files (NEVER `git add -A` inside the submodule). Commit message MUST cite the operator mandate (once the verbatim quote is available) + `Classification: universal`:

```
cd constitution
git add Constitution.md CLAUDE.md AGENTS.md QWEN.md \
        scripts/hooks/guard-forbidden-commands.sh \
        docs/AGENT_GUARDRAILS.md \
        Constitution.html Constitution.pdf CLAUDE.html
        CLAUDE.pdf \
        AGENTS.html AGENTS.pdf QWEN.html QWEN.pdf
git commit -m "feat(§11.4.107): anti-forgetting enforcement
        anchor

Adds the universal PreToolUse guard hook + subagent preamble +
        orchestrator
checklist mandate. Lifts guard-forbidden-commands.sh and
        AGENT_GUARDRAILS.md
to constitution/scripts/hooks/ and constitution/docs/ as
        canonical reference
implementations inherited by reference per §11.4.80.

Classification: universal
Catalogue-Check: reuse constitution/scripts/hooks (new path)
Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>"

# Push to all 4 HelixConstitution upstreams:
git push github main
git push gitlab main
git push gitflic main
git push gitverse main
```

6. **Verify §6.C convergence across all 4 upstreams.** Confirm all 4 remotes report the same HEAD SHA:

```
for remote in github gitlab gitflic gitverse; do
  echo "$remote: $(git -C constitution ls-remote $remote HEAD
        | awk '{print $1}')"
done
# All four must be identical.
```

7. **Bump parent `./constitution` pin + propagate inheritance pointers.** In the Lava parent repo:

```
cd /Users/milosvasic/Projects/Lava
git -C constitution rev-parse HEAD   # capture the new SHA
git add constitution                 # stages the bumped
        submodule pointer
```

The same commit that bumps the pin MUST also:

- Add `§11.4.107` literal (or the compact anchor reference) to every per-scope governance file that needs it per `CM-COVENANT-114-107-PROPAGATION`. In Lava this means: root `CLAUDE.md` (the `§6.AF` block or a new `§6.AF-ext` note), root `AGENTS.md`, root `QWEN.md`, all 17 submodule CLAUDE.md/AGENTS.md/QWEN.md/CONSTITUTION.md, `lava-api-go/CLAUDE.md`, `lava-api-go/AGENTS.md`, `lava-api-go/CONSTITUTION.md`.
- Update `docs/CONTINUATION.md` per §6.S (same commit).
- Run `bash scripts/verify-all-constitution-rules.sh` to confirm the post-bump sweep is clean.

Part C — BLOCKED-UNTIL Status

The clause CANNOT land with a real forensic-anchor quote until the operator pastes the verbatim anti-forgetting mandate.

Per §11.4.6 no-guessing mandate, the forensic anchor MUST be the operator's exact words. The current draft marks the placeholder:

```
UNCONFIRMED: pending operator's verbatim anti-forgetting mandate quote
             (tracked: PENDING item 3 in the current session's CONTINUATION.md handoff)
```

Two options for proceeding:

Option (i) — Land now with the UNCONFIRMED: marker, fill the quote in a follow-up commit.

The clause is structurally complete; the UNCONFIRMED marker is §11.4.6-compliant (it explicitly signals that the quote is unverified). A follow-up commit fills the operator's actual words once provided. This is acceptable because: the hook exists, the preamble exists, the mechanical enforcement is fully specified — the forensic quote is narrative context, not a load-bearing technical constraint.

Steps:

1. Execute the Part B 7-step pipeline with the UNCONFIRMED marker in place.
2. Open a workable item (e.g. `LVA-N`): "Fill §11.4.107 forensic-anchor quote — operator to provide verbatim mandate text."
3. When the operator provides the quote, land a follow-up commit: `docs(§11.4.107): fill forensic-anchor operator mandate quote`.

Option (ii) — Hold until the operator pastes the verbatim mandate.

Do not execute the Part B pipeline. Keep this draft in `docs/superpowers/drafts/`. When the operator provides the quote, update the draft and then execute Part B in full as a single commit.

Recommendation: Option (i). The clause's substance (the hook, the preamble, the enforcement) is fully specified and §11.4.6-honest. The UNCONFIRMED marker tells every reader exactly what is pending. The follow-up is low-risk (docs-only, no gate changes). Landing the clause sooner means the `CM-COVENANT-114-107-PROPAGATION` gate starts enforcing the anchor literal across the fleet, which is the actual value the clause delivers.

End of draft.